CMPT 473
Project Report

Rob Cameron

By
Grantley Kuo(grendlee)
Jea Oh Lee (leejeaoh)
Pritpal Bhamra(pritpalb24)

Git link: https://github.com/pritpalb24/next.js

Dimension 1: Mutation Testing of Next.js Image and Routing Code

Problem

Next.js is one of the most used React frameworks in production, but we have identified two source files of interest that could benefit from mutation testing: the image optimizer in packages/next/src/server/image-optimizer.ts and the client router filter builder in packages/next/src/lib/create-client-router-filter.ts. In Assignment 1 we already saw that Stryker can find real gaps in Next.js tests. That assignment told us that test suites were running a lot of code without checking for mutants. This project looks at two bigger and more safety-related files: one that checks every request to /_next/image, and one that builds the Bloom filter the client router uses to decide which paths are safe to prefetch.

Scope

We picked image-optimizer.ts and create-client-router-filter.ts for three reasons. First, both files are full of if branches, format checks, magic-number reads, and path prefix extraction, which is the kind of code mutation operators can really shake up. Second, both files matter for safety: validateParams is the first line of defence against bad or recursive image requests, and the router filter decides which URLs the client will prefetch. Third, they both already had unit tests in test/unit/image-optimizer/ and test/unit/create-client-router-filter.test.ts, which gave us a good starting point.

Configuration

We added a dedicated Stryker config at dimension1/stryker.conf.json that only mutates the two target files and runs them against a custom Jest config at dimension1/jest.stryker.config.js. Per-test coverage lets Stryker skip tests that don't touch a mutant's line.

Stryker's default mutators replaced math operators, logic operators, comparisons, and string literals, flipped if conditions, emptied arrays and object literals, and deleted block bodies. We saved per-file JSON and HTML reports to dimension1/reports/baseline/ for the before state and dimension1/reports/after/ for the after state.

Baseline Results

The baseline scores in Figure 1 showed the test gap. `create-client-router-filter.ts` had 44 mutants. Only 8 were killed for a score of 18.18%, 19 survived, and 17 were not covered at all. The existing test was a single block that checked one Bloom filter non-collision for three hard-coded paths. Every branch below line 30 the dynamic prefix loop, the interception route handling, the redirect classifier, the Bloom filter build was basically never checked.
`image-optimizer.ts` had 945 mutants. 194 were killed for a score of 20.53%, 159 survived, and 592 had no coverage at all. The existing suite checks detectContentType for common formats, extractEtag,

getMaxAge, and fetchExternalImage size limits, but never touches ImageOptimizerCache.validateParams, getCacheKey, the ICO/ICNS/etc magic-number branches.

Survivors explored

We sorted the 176 survived and 609 not-covered mutants into three buckets.
Branches the tests never touch. By far the biggest cause, about 80% of router filter survivors and 66% of image optimizer survivors. For example, validateParams rejects protocol-relative URLs with a specific error message, but no test ever passed a example.com input, so every mutation of that branch, flipped if, changed string survived.

Weak checks on covered branches. For example, the only test for createClientRouterFilter checks one non-collision case for three hard-coded paths. The test never checks anything that should be in the filter, so mutations that quietly broke the filter still passed. Our fix was to assert on numItems plus the positive paths that should be present.

Equivalent or unreachable mutants. A small leftover, for example, tokens = tryToParsePath(source).tokens on line 57. We flagged these instead of writing brittle tests for them.

A point worth mentioning: Stryker's per-test coverage showed that several covered branches were only covered by accident, a helper function imported by the test happened to run them at module load time. Once we had the per-test report we could move those from the "never run" bucket to the "weak check" bucket, which directly shaped what our new tests needed to do.

New Tests

Based on the root-cause sort above, we wrote two new test files in test/unit/:

- create_client_router_filter_stryker.test.ts, 14 cases covering plain static paths, single-level and multi-level dynamic routes, leading-dynamic edge cases, and more.
- image_optimizer_stryker.test.ts, 59 cases covering detectContentType magic-number branch (JPEG, PNG, GIF, WEBP), and the full ImageOptimizerCache.validateParams flow (missing, array, too-long).

After Results

Figure 2 shows the before and after results.

File	Total Mutants	Killed Before	Killed After	Survived Before	Survived After	No Coverage Before	No Coverage After	Score Before	Score After
create-client-router-filter.ts	44	8	41	19	3	17	0	18.18%	93.18%

image-optimizer.ts	945	194	391	159	152	592	402	20.53%	41.38%
Combined	989	202	432	178	155	609	402	20.42%	43.68%

The router filter file went up by 75.00 percent. The image optimizer file went up by 20 percent, the rest of the gap is mostly around the big untested wrappers like fetchInternalImage, imageOptimizer, and sendResponse, which would need sharp and HTTP mocks. Every no-coverage mutant inside validateParams was killed.

To run the tests to confirm:

node_modules/.bin/jest --config dimension1/jest.stryker.config.js
test/unit/create_client_router_filter_stryker test/unit/image_optimizer_stryker

Risks

Two risks are worth calling out. First, inPlace: true trades safety for speed, a hard interrupt during a Stryker run can leave source files mutated. Second, the image optimizer after-score is still held back by untested wrappers (fetchInternalImage, imageOptimizer, sendResponse) that need sharp and HTTP mocks, a good next step beyond this dimension 1.

Conclusion

Mutation testing did what code coverage numbers could not. It showed that a single-assertion test on createClientRouterFilter left 82% of the logic silently broken, and that validateParams had zero unit coverage at all. Sorting survivors by root cause and writing branch-targeted tests took the combined mutation score from 20.4% to 43.7% and more than doubled the killed-mutant count from 202 to 432. The main takeaway is that statement coverage in Next.js internals often overstates how well the tests would catch real bugs, and mutation testing is a cheap and simple way to make that gap visible and fixable.

DIMENSION 2: TEST DATA ADEQUACY EVALUATION VIA WHITE-BOX METHODS

MOTIVATION

Dimension 1 utilized mutation testing to evaluate the fault-sensitivity of the Next.js test suite. However, if a line of code is never run by any test, mutations on that line are never triggered. The suite can look fine even with these missed cases. Branch coverage can fill that gap. Next.js can have many bugs that live at configuration edges, in error handlers, and at the boundaries of valid input, so uncovered branches are a big risk. Dimension 2 intention is to find which code paths are not tested at all and if it is possible to add tests to reach them.

TARGET MODULES

We measured branch, line, and function coverage on two Next.js source files. These are the same two files studied in Dimension 1.

packages/next/src/server/image-optimizer.ts is the server-side image optimizer. This module manages the server-side image optimization pipeline. It is architecturally complex, containing dense decision trees for MIME-type negotiation, cache-header computation, and multi-step image transformations.

packages/next/src/lib/create-client-router-filter.ts is the module that builds the client-side bloom filter from static paths and redirect paths. This module constructs the Bloom filter used by the client-side router. It contains complex logical branches related to dynamic segment extraction, interception-route normalization, and regex-based path parsing.

METHOD

Coverage was collected with c8 wrapped around Jest. V8's block coverage was mapped back to the TypeScript source files through source maps. We used c8 here because Dimension 1 used it too. We did two runs: a baseline with the 84 Dimension 1 tests, and a second run with the Dimension 2 tests added 64 new tests.

BASELINE COVERAGE

The baseline showed that image-optimizer.ts had 85.57% branch coverage and 74.19% function coverage. create-client-router-filter.ts was higher at 92.85% branch coverage. Combined, the baseline branch coverage was 85.89%. Dimension 1's suite was strong on common paths, but gaps were primarily located in config-gated code and error handlers.

Figure 3

File	Branch	Line	Functions
image-optimizer.ts	85.57% (272/318)	87.78% (1136/1294)	74.19% (33/44)
create-client-router-filter.ts	92.85% (14/15)	100% (74/74)	100% (3/3)
Combined	85.89% (286/333)	88.45% (1210/1368)	75.75% (36/47)

CLASSIFICATION OF UNCOVERED BRANCHES

We looked at uncovered branches in the source and categorized them:

Figure 4

Category	Count	Share	Actionable
Configuration-dependent paths	15	34.1%	yes
Boundary conditions	12	27.3%	yes
Error handlers	5	11.4%	yes
Fallbacks	4	9.1%	yes
Compiled boilerplate	8	18.2%	no

Config-gated branches were the biggest group. Boundary conditions covered edge cases like isIP(hostname). Error handlers covered internal fetches with statusCode === 0 and redirects with no Location header. Fallbacks covered content-type negotiation when the type was not webp or avif. Compiled boilerplate is things like TypeScript's _interop_require_default helpers and lazy module getters. We marked those not-useful because they are compiler output.

IMPROVEMENT RESULTS

After adding the new tests, image-optimizer.ts branch coverage went from 85.57% to 88.99% (+3.42%), and create-client-router-filter.ts went from 92.85% to 93.33% (+0.48%). Combined branch coverage went from 85.89% to 89.18%, a gain of 3.29 points and 29 newly covered branches. Line coverage rose by 4.38 points to 92.83%. Function coverage rose by 17.86 points to 93.61%, a big jump because several of the previously uncovered functions were full utilities with no calls at all, not just isolated branches.

Figure 5

File	Branch	Lines	Functions
image-optimizer.ts	88.99% (283/318)	92.42% (1196/1294)	93.18% (41/44)
create-client-router-filter.ts	93.33% (14/15)	100% (74/74)	100% (3/3)
Combined	89.18% (297/333)	92.83% (1270/1368)	93.61% (44/47)

Metric	Before	After	Change
Branch coverage, image-optimizer	85.57%	88.99%	3.42%
Branch coverage, client-router	92.85%	93.33%	0.48%

Branch coverage, image-optimizer	85.89%	89.18%	3.29%
Line coverage (combined)	88.45%	92.83%	4.38%
Function coverage (combined)	75.75%	90.90%	17.86%

DELIVERABLES

All files for Dimension 2 are under
- `packages/next/src/lib/create-client-router-filter.test.ts`
- `packages/next/src/server/image-optimizer.test.ts`
- `test/c8-coverage/`
- `test/c8-coverage-current/`
- `test/c8-coverage-phase-one/`

DISCUSSION AND LIMITATIONS

Config-gated code represented the largest portion of missed branches. Next.js utilizes numerous policy switches that tests focusing on "standard" behavior frequently overlook. Boundary conditions comprised the second-largest group, mirroring a common trend where tests centered on typical inputs fail to address edge cases. Error handlers ranked third, although they accounted for only 11.4% of the gaps, they remain critical since production incidents often occur within error paths. A test suite that fails to execute a catch block cannot verify if failures are being silently swallowed.

Dimension 2 measures the actual reach of the code. Neither metric independently would have identified the gaps discovered in this study. While mutation testing might indicate high kill rates for reachable code, it would leave the same 34% of policy-gated branches unexamined.

A primary limitation of this research is that 18% of uncovered branches originate from code generated automatically by the TypeScript compiler, such as module getters and import helpers. Because tests designed for the source cannot access this code, achieving 100% coverage under c8 is impossible. Future efforts should involve rerunning the analysis using Jest's Istanbul coverage mode, which provides a more accurate figure by excluding compiler-generated code. Additionally, evaluating a third Next.js middleware file could determine if these gap patterns persist beyond routing filters and image handling.

DIMENSION 3 : PERFORMANCE TESTING OF NEXT.JS RENDERING MODES

PROBLEM

Next.js is widely used in production because it supports several rendering strategies for web applications, mostly server-side rendering (SSR), static site generation (SSG) and incremental static regeneration (ISR). These modes give developers flexibility but they also introduce different runtime costs. In practice, the choice of rendering strategy affects how fast pages respond under load and how much work the server must perform per request. This is especially important in Next.js because rendering performance directly affects user experience and core web vitals.

The proposal for this project identified performance testing as the third major quality dimension. In particular, it stated that SSR, SSG and ISR should be compared using a controlled benchmark application, production mode execution with next build and next start, and load generation with k6. The purpose of this dimension was to measure latency and throughput and determine which rendering strategy performs best under increasing demand.

SCOPE

We focused on the three main rendering methods exposed by Next.js in production:

- SSR, implemented with getServerSideProps
- SSG, implemented with getStaticProps
- ISR, implemented with getStaticProps together with a revalidate interval

We did not include development mode in the final measurements because it does not reflect real deployed behaviour and includes extra compilation and debugging overhead. This follows the original proposal, which explicitly placed development mode out of scope for performance testing. Instead, all benchmark results reported here were collected using a production build and a production server.
The scope of the benchmark was intentionally narrow. Rather than trying to benchmark the entire Next.js framework or all of its internal subsystems, we built a small benchmark application with one page per rendering mode. This allowed us to isolate the effect of the rendering strategy itself while keeping the surrounding environment controlled and comparable across tests.

BENCHMARK APPLICATION

A benchmark application was created inside the repo for dimension 3, it contains three pages:

- pages/ssr.tsx
- pages/ssg.tsx
- pages/isr.tsx

Each page renders a list of generated items so that the server does measurable work before returning the response. The SSR page generates the content on every request. The SSG page generates the content at build time. The ISR page generates static content with periodic regeneration using a 10 second revalidation interval.

This design follows the benchmark-app approach described in the proposal. The proposal stated that one SSR page, one SSG page, and one ISR page should be built and then exercised under load to compare runtime performance.

METHOD

We used k6 to load test the three benchmark routes separately. The application was first built and served in production mode using:

```
pnpm build
pnpm start
```

Then ran a k6 script against each route one at a time:

- /ssr
- /ssg
- /isr

The script ramped virtual users from 1 to 50 over five stages. For each request, the script checked that the response returned HTTP 200 and that the body was non-empty. It also recorded custom latency metrics as well as the built-in HTTP timing metrics from k6.

The main metrics we collected were:

- Average latency
- Median latency
- P90 latency
- P95 latency
- P99 latency
- Maximum latency
- Total requests
- Requests per second
- Request failure rate
- Success rate

Before running k6, we also performed a small manual sanity check in development mode to verify that the pages behaved as intended. SSR changed on very refresh, SSG stayed fixed until rebuild and ISR updated after the revalidation period.

RESULT

The results are shown below.

Figure 6

Rendering Mode	SSR	SSG	ISR
Avg Latency	8.78 ms	3.75 ms	4.80 ms

Median	8.15 ms	3.72 ms	4.42 ms
p90	13.26 ms	5.09 ms	7.10 ms
p95	13.83 ms	5.62 ms	8.52 ms
p99	15.60 ms	6.71 ms	12.88 ms
Max	28.89 ms	8.67 ms	21.04 ms
Requests	2303	2313	2311
Req/s	17.64/s	17.69/s	17.66/s
Failed Requests	0.00%	0.00%	0.00%
Success Rate	100.0%	100.0%	100.0%

The latency ranking was:

1. SSG fastest
2. ISR second
3. SSR slowest

All three routes passed the configured thresholds. Each mode had a 100.0% success rate and a 0.00% failed request rate. All p95 and p99 values were far below the benchmark threshold.

DISCUSSION

SSG performed best overall. This makes sense because the page is pre-generated at build time so the server mostly serves already prepared output. There is little runtime work compared with the other two modes. Its average latency was only 3.75 ms, and its p95 latency was 5.62 ms, which were the lowest values.

ISR performed slightly worse but much better than SSR. This also makes sense. ISR behaves like static generation for most requests, which keeps latency low but it adds regeneration logic through revalidation. That extra mechanism likely explains why its tail latency was higher than SSG even though it remained much faster than SSR.

SSR was the slowest rendering mode. This is the most intuitive result because SSR must perform rendering work on every request rather than serving a build-time artifact. Its average latency was 8.78 ms and its p95 latency was 13.83 ms, both clearly above SSG and ISR.

One interesting result is that the throughput values were very close across all three modes, all around 17.6 requests per second. The reason is likely the benchmark script itself. Our k6 script included a one second sleep per iteration which limits how aggressively each virtual user can issue requests. Because of that, throughput should be treated more as a consistency check than the main differentiation metric. The latency values are the more informative part of the experiment we did.

The benchmark therefore supports the general architectural tradeoff that Next.js exposes:

- SSG is best when maximum speed is the priority and content can be generated ahead of time.

- ISR is a compromise when freshness is needed but full SSR cost is undesirable.
- SSR is the most flexible for dynamic data but comes with the highest request time cost.

These results are very meaningful because they show that rendering mode in Next.js is not just a design preference. It has a measurable effect on performance even in a small controlled benchmark.

LIMITATIONS

There are several limitations to this study,

First, the benchmark application is intentionally simple. It isolates the effect of the rendering mode well, but it does not represent the full complexity of a large production Next.js application with middleware, database access, caching layers or external API calls.
Second, the benchmark was run in a local environment, which reduces outside network noise but does not fully capture deployment conditions. Real hosted environments could introduce additional latency from infrastructure, CDN behaviour or cold starts.
Third, the k6 script included a sleep(1) in each iteration. This makes the benchmark easier to control and compare but it limits raw throughput and compresses differences in request rate. A more aggressive script without per-iteration sleep would likely show larger throughput separation between rendering modes.
Fourth, although the proposal also mentioned profiling and flame graphs, this part was not fully developed in the current dimension. The latency results already distinguish the three rendering strategies, but a next step would be to capture Node.js CPU profiles or flame graphs to connect the observed delays back to specific rendering functions or framework internals.

CONCLUSION

Dimension 3 evaluated the performance of the three main rendering strategies used in Next.js : SSR, SSG, and ISR. using a benchmark application, production-mode execution and k6 load testing, we measured latency and throughput while ramping from 1 to 50 virtual users. The result showed that SSG was the fastest, ISR was second and SSR was the slowest. All three modes were stable and passed the configured threshold, but the differences in latency demonstrates that rendering strategy has real performance cost.

The main point to understand is that Next.js rendering choice have measurable runtime consequences. Static generation provides the best response times, incremental regeneration offers a middle ground between speed and freshness and finally server-side rendering remains the most expensive option because it performs work on every request. This confirms the motivation in the proposal and shows that performance testing is a useful complement to the mutation and white-box adequacy analysis from the other two project dimensions.